

Organización de Datos

Trabajo Práctico N°3

Por:

Gabriel Seneca, Mario Arriaga, Mia Casanovas Di Sera y Augusto
Sulsente

Parte 1:

1)

El archivo podría verse de la siguiente forma:

Posición	Dato
0	<BASURA>
48	35175
187	7993
252	24971
546	21362
709	8515
817	50255
1061	10168
1062	53101
1070	10177
1081	32305
1164	29786
1192	12901
1210	32434
1299	<BASURA>
<eof>	<eof>

- La cantidad máxima de claves estará dada por el tamaño del archivo, en este caso 1300 claves organizadas desde la posición 0 hasta la 1299.
- El factor de ocupación será:
$$\text{Registros Ocupados} / \text{Registros Totales} = 14/1299 \sim 0,011$$
- Si, existieron colisiones, las cual fué con los valores 10168 y 53101, ya que
$$f(10168) = f(53101) = 1061$$
- Las colisiones se solucionaron utilizando el método de “Progressive Overflow”

2)

Método 1: Función de Multiplicación

$$f(x) = \text{piso}(200 * \{x * (\sqrt{5} - 1)/2\})$$

Posición	Dato
0	<BASURA>
33	10168
44	53101
51	12901
59	50255
62	32434
69	35175
88	21362
111	8515
117	32305
146	10177
152	29786
185	24971
189	7993
199	<BASURA>
<eof>	<eof>

Método 2: Cuadrado Medio

$$f(x) = x^2 = XXXyyyXXX$$

Posición	Dato
0	<BASURA>
5	29786
35	12901
88	7993
105	8515
128	35175
135	21362
150	24971
156	50255
161	32305
171	10177
172	53101
188	10168
196	32434
199	<BASURA>
<eof>	<eof>

3)

Posición	Dato
0	<BASURA>
24	vodka
32	ojota
34	limon
215	oh
326	caradura
327	caradura
328	ojo
427	gato
511	<BASURA>
<eof>	<eof>

Aclaración: Elegimos como número primo 509 ya que aunque la operación de forma directa no caerá en 510 y/o 511, los espacios igual serán aprovechados al utilizar Progressive Overflow, lo cual creemos mucho mejor que utilizar como número primo 521 y generar 9 colisiones extra.

1) Suponga un sistema para la gestión de una maratón de 42Km. El cupo máximo es de 12000 corredores. En la inscripción cada corredor provee: número de DNI, nombre, sexo, edad, categoría. Hay un último campo que es el tiempo (que se carga al finalizar la carrera).

Para la solución se deberá usar un archivo directo, donde deberá diseñar una función hash que mapee número de DNI a posición en el archivo (que es el número de corredor otorgado en el momento de la inscripción).

Implemente: función hash, archivo, altas, modificaciones, bajas. Carga de tiempos. Listados de tiempos general y por categoría.

La solución consta de un archivo maestro que en un principio, pre carrera, contendrá DNI, nombre, sexo, edad, categoría, borrado (1 o 0). Post carrera se van a cargar los tiempos, se le va a sumar el campo tiempo.

DNI	Nombre	Sexo	Edad	Categoría	Borrado	Tiempo

La clave primaria de la tabla es el DNI. Aplicamos el método de hashing Multiplicación, si hay colisiones, estas serán solucionadas mediante Progressive Overflow.

Implementación de función hash:

```
int HashMultiplicacion(unsigned int_8 k) {  
  
    double parte_fraccionaria, parte_entera;  
    A = 0.6180339887  
    word M=12007;  
    double x;  
    x = k * A;  
    parte_fraccionaria = x - parte_entera;  
    x =* M;  
    x = floor(x);  
  
    return x;  
  
}
```

archivo donde se encuentran los tiempos de cada corredor, luego de haber finalizado la carrera. (Carga de tiempos).

Dirección física	DNI (clave)	Tiempo
	44860507	

Implementacion de Carga de tiempos / Bajas:

```
void CargaTiempos (TArchivo archivo_Maestro*,string NomArchivo,  
TArchivoTiempos* ArchivoTiempos) {  
    word M=12007;  
    unsigned int_8: DNI;  
    archNuevo:ArchivoNuevo;  
    regTiempos:TRegTiempos;  
    reg:TregistroInscriptos;  
    pos:word;  
    regNuevo: TRegNuevo \contiene el contenido de los inscriptos pre carrera +  
el tiempo post carrera  
  
    Assign(archNuevo, "Archivonuevo.dat");  
    Reset(archNuevo)
```

```

Reset(archTiempos)
fread(&regTiempos,sizeof(regTiempos),1,archTiempos);
while (!eof(archTiempos)) {
    pos=HashMultiplicacion(reg.DNI);
    seek(pos,Arch_Maestro);
    fread(&reg,sizeof(reg),1, Arch_Maestro);
    if (regTiempos.DNI!=reg_Original.DNI){ \les el mismo corredor
        *cargo las variables DNI, nombre, sexo, edad, categoria, tiempo
a RegNuevo
        fwrite(regNuevo,sizeof(regNuevo),1,archNuevo);
    }
    else {
        encontrado=0;
        while (!encontrado && !eof(arch_Original))do {
            pos= (pos+1)mod M;
            seek(pos,ArchMaestro);
            fread(&reg,sizeof(reg),1, Arch_Maestro);
            if (regTiempos.DNI==reg_Original.DNI)
                encontrado=1;
        }
        if (!encontrado) {
            Reset(archOriginal);
            while (!encontrado && !eof(arch_Original))do {
                pos= (pos+1)mod M;
                seek(pos,ArchMaestro);
                fread(&reg,sizeof(reg),1, Arch_Maestro);
                if (regTiempos.DNI==reg_Original.DNI)
                    encontrado=1;
            }
            if (!encontrado)
                printf("error en la carga de datos /n");
            else {
                *cargo las variables DNI, nombre, sexo, edad,
categoria, tiempo a RegNuevo
                fwrite(regNuevo,sizeof(regNuevo),1,archNuevo);
            }
        }
    }
    else {
        *cargo las variables DNI, nombre, sexo, edad, categoria,
tiempo a RegNuevo
        fwrite(regNuevo,sizeof(regNuevo),1,archNuevo);
    }
}
fread(&regTiempos,sizeof(regTiempos),1,archTiempos);
}

close(ArchTiempos);
Close(archMaestro);

```

```

Close(ArchNuevo);
Delete(ArchMaestro);
Rename(ArchNuevo, NomArchivo) \\contiene el nombre del archivo maestro
}

```

Esta función se encarga de cargar el tiempo de los corredores. Va a leer el archivo Tiempos, en el cual se encuentran los DNIs, clave primaria del archivo maestro, y realiza un hashing para encontrar la celda del corredor en cuestión. Si la encuentra hace un seek a la celda del archivo maestro y carga los datos de la persona (contenidos en ese archivo) y su tiempo (contenido en el archivo de tiempos) en el nuevo archivo, que va a pasar a ser el archivo maestro. Si no la encuentra, sigue buscando la celda hasta que recorra todo el archivo maestro y verifique que no existe, de ser así, se muestra un cartel que dice “error en la carga de datos”, ya que todo quien corrió la carrera debió estar inscrito.

Consideramos que al hacer esto también se realizan las bajas, ya que en el archivo nuevo quedarán los datos de las personas que efectivamente corrieron la carrera. También se podrán realizar las bajas mediante una función que busque la celda en el archivo maestro, puede encontrarla o no, si la encuentra, va a marcar esa celda como borrada, sino, no hace nada (solo muestra que no la encontró).

Implementación de altas y modificaciones:

```

void AltasYMod(TArchMaestro * archMaestro) {
    word M=12007;
    TArchMaestro archAltasyModificacones;
    regAltas:TRegInscriptos;
    word pos;
    if (archMaestro = fopen("archMaestro", "rb+")==NULL)
        error en la apertura del archivo maestro
    else {
        if (archAltas = fopen("archAltasyModificaciones", "rb")==NULL)
            error en la apertura del archivo altas
        else {
            while
(fread(&regAltas,sizeof(regMaestro),1,archAltasyModificaciones)==1)do
            {
                pos=HashMultiplicacion(regAltas.DNI);
                seek(pos,ArchMaestro);

                fread(&regMaestro,sizeof(regMaestro),1,archMaestro);
                if (regMaestro==NULL || regMaestro==borrado)
                    inserto en archivo maestro
                else {
                    if (regMaestro.DNI==regAltas.DNI) {
                        genero la modificación
                    }
                    else
                    {

```



```

        while ((regMaestro!=NULL ||
regMaestro!=Borrado || regMaestro.DNI!=regAltas.DNI)
&& !eof(archMaestro))
            pos= (pos+1)mod M;
            seek(pos,ArchMaestro);
            fread(&reg,sizeof(reg),1,
Arch_Maestro);
        }
        if (eof(archMaestro)
            printf("no hay más cupos abiertos");
        else
            if (regMaestro==NULL || regMaestro==Borrado)
                inserto registro en archivo maestro;
            else
                GeneroModificacion;
        }
    }
    Close(archAltas);
}
Close(archMaestro);
}
}

```

Lo que hace esta función es agregar a las personas que quieren inscribirse en la carrera en el sistema, y realizar modificaciones de los ya inscritos. Tenemos un archivo de altas Y modificaciones y el archivo maestro que contiene la información de los ya inscritos. Lo que hace la función es leer el archivo de altas, hace un hashing de multiplicación con el DNI de la persona y busca en la dirección física del archivo maestro a ver si esa celda está ocupada o borrada, para agregar la información de la persona que intenta inscribirse. Si esa celda ya contiene los datos de una persona inscrita, distinta a la de la persona en cuestión, se produce colisión y se busca otra celda mediante Progressive Overflow para evitar desbordamiento; a su vez, también se comparan los DNIs continuamente para ver si encontramos una celda que contiene los datos de la persona que intenta modificar su información. La función no sabe si se quiere modificar o dar de alta a una persona. Mediante hashing busca la dirección física donde debería encontrarse su celda según su clave primaria (DNI), si esa celda ocupada contiene su información, la modifica con la nueva, y si está vacía, significa que quiere inscribirse, y lo inscribe. Si está ocupada y no con su información, sigue buscando un espacio, si recorrió todo el archivo y no encontró ninguna celda libre, muestra un mensaje diciendo que ya no hay más cupos libres para inscribirse.

Implementación del listado de clasificaciones generales y por categorías:

La implementación del listado de clasificaciones generales sería void, recorreremos el archivo completo secuencialmente para mostrar por pantalla a cada corredor, si la celda no es nula o no está marcada como borrada se muestra, sino no se muestra.

Por otro lado, para mostrar el listado por categorías, teniendo en cuenta que el archivo no va a estar ordenado por categorías ya que el orden depende del hashing, se deberá realizar la función por cada categoría existente. Se va a recorrer el archivo secuencialmente, en un ciclo while !eof(archivo maestro), se lee el registro, se pregunta si la celda está vacía o no. Si está vacía se saltea, vuelve a comenzar el ciclo y lee. Si no está vacía, verifica mediante una variable que contenga la categoría a mostrar si la persona pertenece a esa categoría. De ser así, la muestra. Y sigue el ciclo.

¿Es eficiente esta solución al momento de imprimir los listados de clasificaciones generales y por categorías? ¿Usaría la misma solución si la toma de tiempo se hace de forma electrónica con sensores tipo RFID en la largada, media maratón (km 21) y final (Km 42)? ¿Qué cambiaría?

No es eficiente si se quiere imprimir el listado por categorías, ya que el archivo no va a estar ordenado por categoría, se va a tener que recorrer el archivo completo la cantidad de veces de categorías existentes. Por otro lado, no es necesariamente ineficiente si se trata de imprimir el listado de clasificaciones. La ineficiencia depende de la cantidad de inscritos que hay, y de su dispersión. Si hay pocos inscritos y muy dispersos, se van a leer muchas celdas vacías innecesariamente, si hay pocos y de casualidad quedaron en las primeras 2000 celdas, se van a leer otras 10007 innecesariamente.

2) Suponga un sistema para el manejo de personal de una empresa. Los datos a almacenar son apellido y nombre, dirección, teléfono, y fecha de ingreso. Para la solución se plantea un archivo directo donde el apellido y nombre es la clave de acceso. Implemente una función hash, creación de archivo principal, alta, modificaciones, bajas. El manejo de colisiones se debe resolver por área separada de overflow.

cada registro va a tener la siguiente estructura:

Apellido	Nombre	Dirección	Teléfono	Fecha de ing

Implementacion de la función hash:

```
int valor,tot;
```

```
int HashFoldAdd(string nombre,string apellido){
```

```
    M=500
```

```
    longitud=length(nombre+apellido)
```

```
    cad= nombre+apellido
```

```
    for (int i=0;i<longitud;i++){
```

```
        valor=charaAscii(cad[i])    // guarda el valor de ascii del carácter i
```

```
        tot=tot+valor
```

```

    }
    return tot mod M
}

```

Con esta función se va a recibir un nombre y apellido, el cual se va a fusionar para crear una dirección en la cual se va a guardar en el archivo.

Se van a crear dos archivos, uno que va a tener espacio para 500 registros, que sería el archivo principal y un archivo para manejar el overflow. (área separada de overflow)

La forma en la que funciona es la siguiente, supongamos que tenemos dos claves que tienen igual H, por lo tanto deberían guardarse en la misma posición. Para solucionar este problema, cada elemento va a tener un puntero hacia otro elemento que tenga el mismo H, el cual va a ser almacenado en el área separada de overflow.

```

void crear_archivos() {
    // overflow vacío
    f = fopen("Overflow.dat", "wb");
    fclose(f);

    FILE *f = fopen("Principal.dat", "wb")
    M=500
    Empleado e
    // tiene por campos nombre,apellido,dirección,teléfono y fecha de ingreso

    for (int i = 0; i < M; i++){
        cargaempleado(&e)
        fseek(f,HashFoldAdd(e.nombre,e.apellido),SEEK_SET)
        if (espaciodisponible)
            fwrite(&e, sizeof(Empleado), 1, f)
        else
            guardoenoverflow(e)
    }
    fclose(f);
}

```

dónde espaciodisponible es una función que devuelve verdadero si el lugar de memoria está vacío o tiene un elemento borrado, y falso si el lugar está ocupado, lo cual causa que se vaya a guardar el valor en el área de overflow.

Alta:

```
void alta(Empleado e){
    Empleado e2
    FILE *f = fopen("Principal.dat", "wrb")
    fseek(f,HashFoldAdd(e.nombre,e.apellido),SEEK_SET)
    if (espaciodisponible)
        fwrite(&e, sizeof(Empleado), 1, f)
    else{
        fread(&e2,sizeof(Empleado),1,f)
        fwrite(&e, sizeof(Empleado), 1, f)
        guardoenoverflow(e2->sig,e)
    }
}
```

este pseudocódigo recibe un empleado a cargar, le hace su hash y verifica si en esa posición hay espacio disponible. Si lo hay lo escribe. Si no lo hay lo guarda en la sección de overflow.

La función guardoenoverflow se encarga de verificar toda la ruta que sigue e2->sig, para verificar todos los registros que tienen el mismo hash, desde ahí va a ir al final de la lista enlazada y va a agregar el nuevo registro e.

Modificaciones:

```
void modifica(string nombre,string apellido){

    FILE *f = fopen("Principal.dat", "wrb")
    fseek(f,HashFoldAdd(nombre,apellido),SEEK_SET)
    if (existeElemento){
        //hace las modificaciones
    }
    else
        if (estaenoverflow)
            //hace modificaciones
        else
            error, el registro no existe

}
```

Esta función modifica abre el archivo, y verifica si el elemento existe en el archivo principal, si no existe se verifica si está en el overflow, y si tampoco está en el overflow significa que el nombre y apellido ingresado no corresponde con ningún registro ingresado.

```

void Bajas(string nombre,string apellido){
    FILE *f = fopen("Principal.dat", "wrb")
    fseek(f,HashFoldAdd(nombre,apellido),SEEK_SET)
    if (existeElemento){
        // hace la baja ya sea física o lógica
    }
    else
        if (estaenoverflow)
            // hace la baja en el overflow ya sea física o lógica
        else
            error, el registro no existe
}

```

Esta función Bajas se encarga de abrir el archivo, y verificar si lo encuentra. En base a eso se realiza la baja ya sea física(vacía el registro y en este caso trae el elemento siguiente de la lista al archivo principal) o lógica(se marca el registro para que al momento de buscar se ignore)

Si no está en ninguno de los dos da error.